

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій



Лабораторно робота №6
з дисципліни: “Технології доповненої реальності”
на тему:
“Розпізнавання зображень: AR на основі маркерів”

Виконав:
ст. групи ПП-44
ВЕРЕЩАК Богдан

Прийняв:
ас. Юрій ПАТЕРЕГА

Львів - 2026

Мета роботи

Після виконання лабораторної роботи студент зможе:

- пояснити принцип роботи image tracking в AR;
- створювати Reference Image Library в Unity;
- налаштовувати AR Tracked Image Manager;
- прив'язувати різний 3D-контент до різних зображень;
- обробляти події додавання, оновлення та видалення tracked images.

Індивідуальне завдання

$$N = 3, P = 7$$

$$V = ((N - 1 + (P - 1) \times 2) \% 15) + 1 = (14 \% 15) + 1 = 15$$

$$M = ((N - 1 + P) \% 4) \rightarrow 0 = A, 1 = B, 2 = C, 3 = D, \quad M = 1 = B$$

М	Тематика	Приклад маркерів	Контент
В	Вікторіанський живопис	Картини Тернера, Росетті, Мілле, Ватерхауса	Ім'я художника, рік, стиль, 3D-рамка

V	Завдання	Підказка
15	Анімація появи контенту (масштабування від 0 за 0.5с) + анімація зникнення (масштабування до 0 при втраті). Плавні переходи	StartCoroutine, Mathf.Lerp для scale 0→1 (added) та 1→0 (removed)

Хід роботи

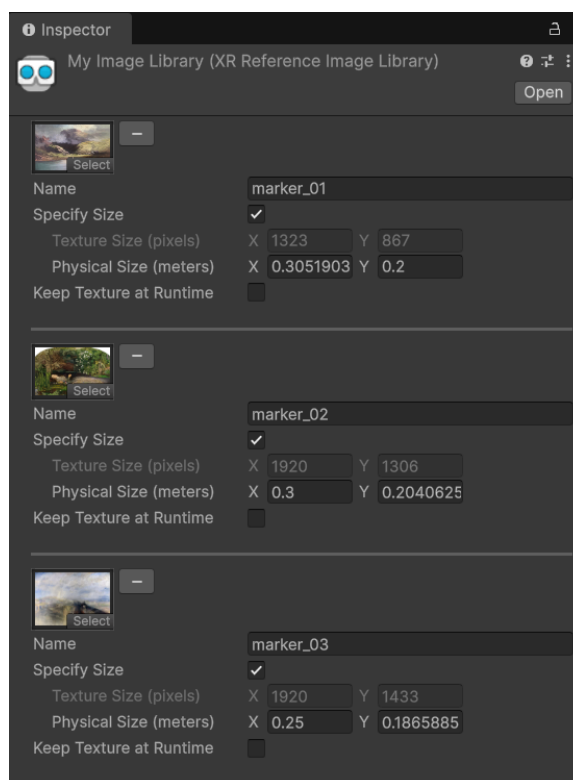


Рис. 1. Налаштований Reference Image Library

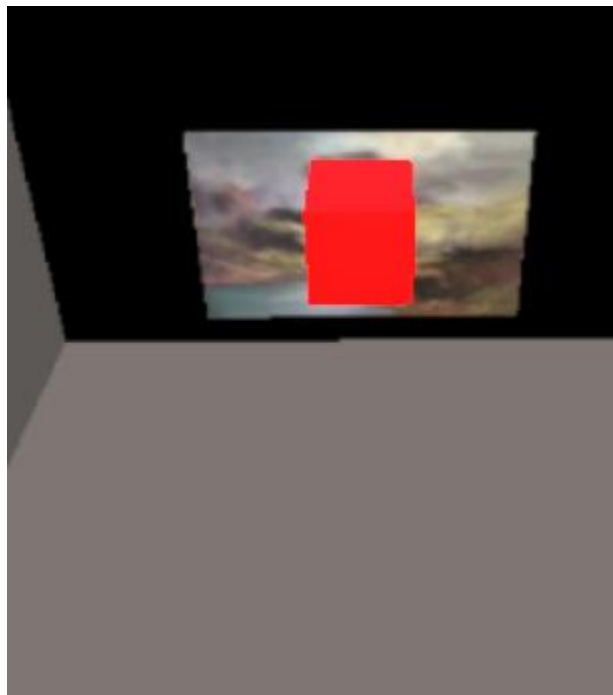


Рис. 2. Базовий трекінг marker 1 загального завдання



Рис. 3. Базовий трекінг marker 2 загального завдання

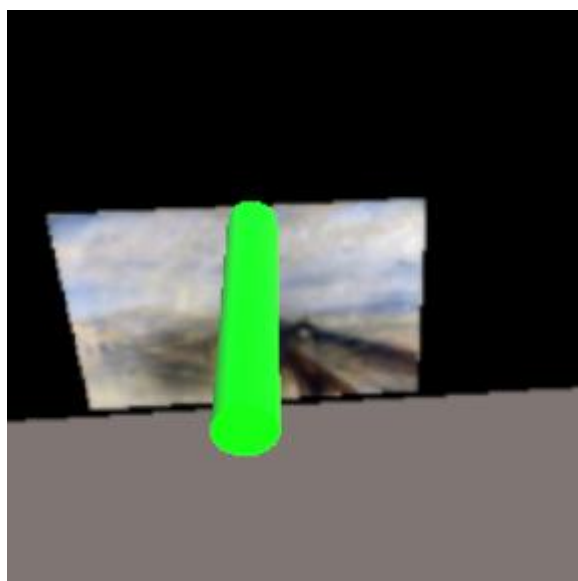


Рис. 4. Базовий трекінг marker 3 загального завдання

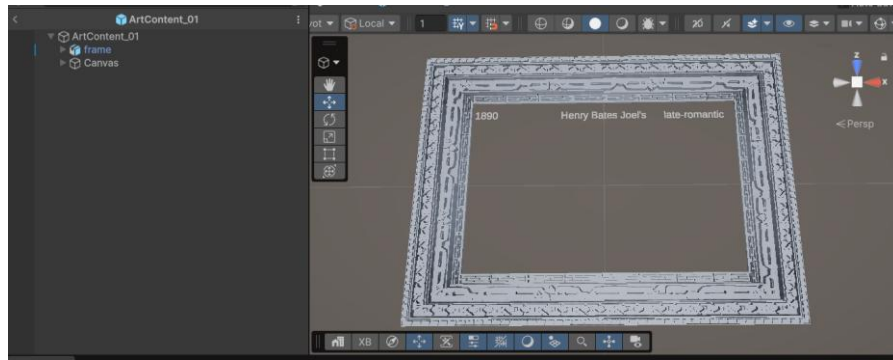


Рис. 5. Створений контент

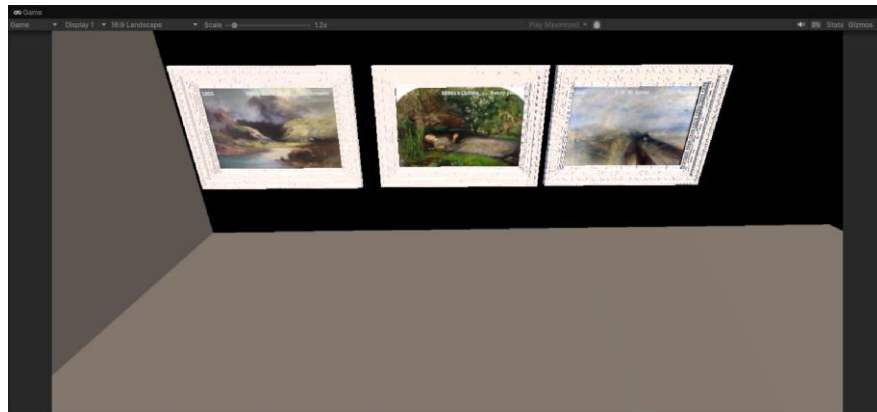


Рис. 6. Відображення на маркері

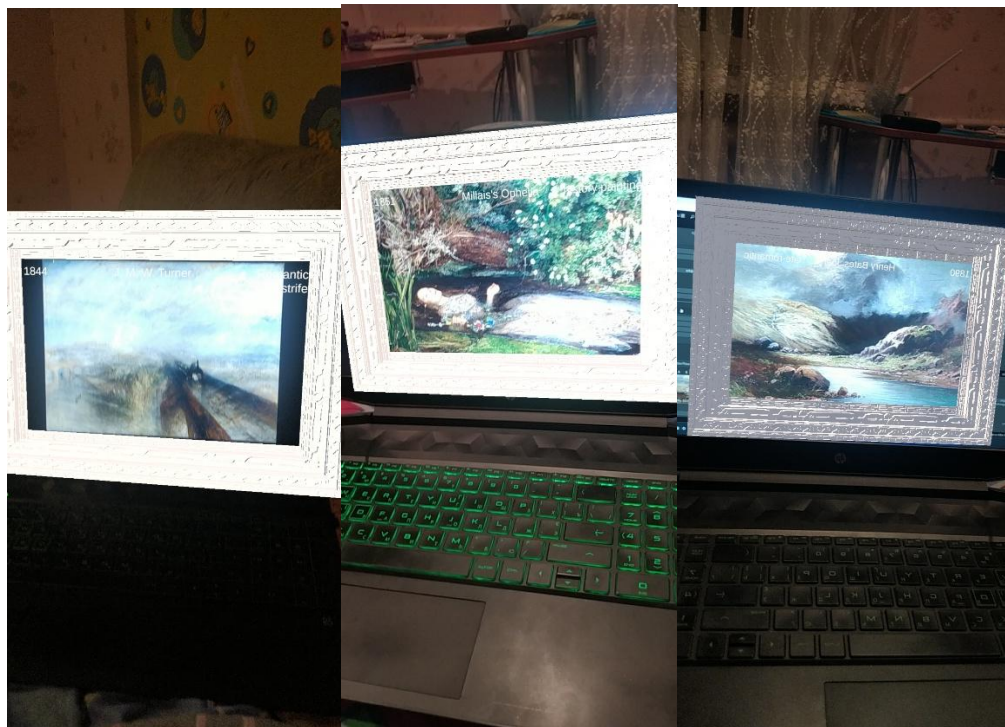


Рис. 7. Вигляд на мобільному пристрої

Код індивідуального завдання (зміни від початкового коду вказані жирним курсивом)

ImageTracker.cs:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
```

```

using UnityEngine.XR.ARSubsystems;

public class ImageTracker : MonoBehaviour
{
    [Header("AR Components")]
    [SerializeField] private ARTrackedImageManager imageManager;

    [Header("Content Prefabs")]
    [Tooltip("Prefab для кожного зображення. Element 0 = marker_01, Element 1 = marker_02...")]
    [SerializeField] private GameObject[] contentPrefabs;

    [Header("Animation Settings")]
    [SerializeField] private float animationDuration = 0.5f;

    private Dictionary<string, GameObject> spawnedContent = new Dictionary<string, GameObject>();
    private Dictionary<string, Vector3> originalScales = new Dictionary<string, Vector3>();
    private Dictionary<string, Coroutine> activeCoroutines = new Dictionary<string, Coroutine>();
    private Dictionary<string, bool> trackingStates = new Dictionary<string, bool>();

    void OnEnable()
    {
        if (imageManager == null)
        {
            Debug.LogError("ImageTracker: Image Manager не призначено в Inspector!");
            return;
        }
        imageManager.trackedImagesChanged += OnTrackedImagesChanged;
    }

    void OnDisable()
    {
        if (imageManager != null)
            imageManager.trackedImagesChanged -= OnTrackedImagesChanged;
    }

    private void OnTrackedImagesChanged(ARTrackedImagesChangedEventArgs args)
    {
        foreach (var trackedImage in args.added) HandleAddedImage(trackedImage);
        foreach (var trackedImage in args.updated) HandleUpdatedImage(trackedImage);
        foreach (var trackedImage in args.removed) HandleRemovedImage(trackedImage);
    }

    private void HandleAddedImage(ARTrackedImage trackedImage)
    {
        string imageName = trackedImage.referenceImage.name;
        int index = GetImageIndex(imageName);

        if (index < 0 || index >= contentPrefabs.Length) return;

        GameObject content = Instantiate(
            contentPrefabs[index],
            trackedImage.transform.position,
            trackedImage.transform.rotation,
            trackedImage.transform
        );
    }
}

```

```

string key = imageName ?? trackedImage.referenceImage.guid.ToString();
spawnedContent[key] = content;

Vector3 originalScale = content.transform.localScale;
originalScales[key] = originalScale;
content.transform.localScale = Vector3.zero;

if (trackedImage.trackingState == TrackingState.Tracking)
{
    RunScaleRoutine(key, content, originalScale, true);
}
}

private void HandleUpdatedImage(ARTrackedImage trackedImage)
{
    string imageName = trackedImage.referenceImage.name;
    string key = imageName ?? trackedImage.referenceImage.guid.ToString();

    if (!spawnedContent.TryGetValue(key, out GameObject content)) return;

    bool isTracking = trackedImage.trackingState == TrackingState.Tracking;
    if (trackingStates.ContainsKey(key) && trackingStates[key] == isTracking)
    {
        return;
    }

    trackingStates[key] = isTracking;
    Vector3 targetScale = isTracking ? originalScales[key] : Vector3.zero;

    RunScaleRoutine(key, content, targetScale, isTracking);
}

private void HandleRemovedImage(ARTrackedImage trackedImage)
{
    string imageName = trackedImage.referenceImage.name;
    string key = imageName ?? trackedImage.referenceImage.guid.ToString();

    if (spawnedContent.TryGetValue(key, out GameObject content))
    {
        content.transform.SetParent(null);

        StartCoroutine(DestroyAfterAnimation(content));

        spawnedContent.Remove(key);
        originalScales.Remove(key);
        trackingStates.Remove(key);
        if (activeCoroutines.ContainsKey(key)) activeCoroutines.Remove(key);

        Debug.Log($"Image lost: '{imageName}', playing destroy animation");
    }
}

private void RunScaleRoutine(string key, GameObject content, Vector3 targetScale, bool isTracking)
{

```

```

    if (activeCoroutines.ContainsKey(key) && activeCoroutines[key] != null)
    {
        StopCoroutine(activeCoroutines[key]);
    }

    if (isTracking && !content.activeSelf)
    {
        content.SetActive(true);
    }

    activeCoroutines[key] = StartCoroutine(ScaleRoutine(content, targetScale, !isTracking));
}

private IEnumerator ScaleRoutine(GameObject target, Vector3 targetScale, bool
deactivateOnComplete)
{
    Vector3 initialScale = target.transform.localScale;
    float timeElapsed = 0f;

    while (timeElapsed < animationDuration)
    {
        target.transform.localScale = Vector3.Lerp(initialScale, targetScale, timeElapsed /
animationDuration);
        timeElapsed += Time.deltaTime;
        yield return null;
    }

    target.transform.localScale = targetScale;
    if (deactivateOnComplete)
    {
        target.SetActive(false);
    }
}

private IEnumerator DestroyAfterAnimation(GameObject target)
{
    Vector3 initialScale = target.transform.localScale;
    float timeElapsed = 0f;

    while (timeElapsed < animationDuration)
    {
        target.transform.localScale = Vector3.Lerp(initialScale, Vector3.zero, timeElapsed /
animationDuration);
        timeElapsed += Time.deltaTime;
        yield return null;
    }

    Destroy(target);
}

private int GetImageIndex(string imageName)
{
    if (string.IsNullOrEmpty(imageName)) return 0;

    if (imageName.StartsWith("marker_"))

```

```

{
    string numberStr = imageName.Replace("marker_", "");
    if (int.TryParse(numberStr, out int number)) return number - 1;
}

if (imageManager != null && imageManager.referenceLibrary != null)
{
    var library = imageManager.referenceLibrary;
    for (int i = 0; i < library.count; i++)
    {
        if (library[i].name == imageName) return i;
    }
}
return 0;
}
}

```

Відео демонстрація екрану телефону розробленого AR додатку

<https://drive.google.com/file/d/1FDgsx2Fz8GvCW0dXeK3ZrHTiMfEH0mh4/view?usp=sharing>

Висновок

На даній лабораторній роботі я зрозумів принцип роботи image tracking в AR, навчився створювати Reference Image Library в Unity, налаштовувати AR Tracked Image Manager, прив'язувати різний 3D-контент до різних зображень, обробляти події додавання, оновлення та видалення tracked images.